

RIT HMS — System, Architecture, Deployment & Backlog

RIT HMS — System Overview

What it is: a multi-tenant, cloud-native rewrite of the legacy SMART_HMS hospitality management system (POS + back office for Sri Lankan restaurants/hotels), rebuilt as **ASP.NET Core 8 + PostgreSQL + Next.js 15**.

Status: feature-parity-plus with the legacy for day-to-day operations. Live test deployment at <https://hms.retailit.lk>.

Compiled from the current code surface (49 SQL migrations, 22 API feature modules, ~21 web screens). Items are graded  **built**,  **v1 / partial**,  **not built**,  **won't-do**.

1. Feature inventory

1.1 Point of Sale (POS)

Feature	Status
Order/tab lifecycle (open → confirm → settle/void), resume open tabs, deep-link	
Touch terminal: category/product grid, search (OS keyboard), keypad, open/custom items	
Modifiers / add-ons, serving-size variants, per-product kitchen routing	
Bill discounts (amount / %) + role-capped with manager-PIN override	
Multi-tender settlement: cash, card, Uber Eats / PickMe prepaid, credit (A/R) , loyalty points	
Tables / floor: per-outlet tables, occupancy, table picker, reservations	
Split / merge bills + table transfer + item-level split	
Full-screen mode, sign-out, branch switch (admin)	
Receipt print (80mm) + reprint of a settled bill	
Covers (adults/children)	
Steward/waiter tips, multi-currency tender, tour-agent commission	

1.2 Kitchen Display (KDS / KOT)

Feature	Status
Live ticket board by station (Hot Kitchen / Bar / Dessert) + counts	
New → Preparing → Ready → Served lifecycle, overdue timers, recall-last-10, PAID badge	
Printer-station routing per product	
All-day item counts, course firing, expo screen, prep-time analytics	

1.3 Menu & catalogue

Products CRUD + CSV import, multi-level categories, units of measure (+ conversion), modifiers, serving-size variants, price levels, per-location pricing, per-product tax class. 

1.4 Tax / VAT (Sri Lanka)

Configurable compound charge engine (Service Charge → SSCL → VAT, per-order-type), compliant tax invoices (sequential numbering, BR/VAT/SVAT, bill branding), VAT-return report. 

1.5 Supply chain

Suppliers, Purchase Orders (approval workflow, currency/terms), GRN (purchase-unit→stock-unit conversion, line discounts/landed cost, free items, supplier invoice no., void/reverse), weighted-average costing, supplier returns (PRN + input-VAT reversal), inter-location transfers, wastage, stock adjustments, **physical stock count** (sheet → variance → on-hand). ✓

1.6 Production

Recipes/BOM with UoM conversion, custom/ad-hoc production, draft→post→void, multi-product notes, multiple recipes per output unit, inter-location production, weighted-avg cost roll-up. ✓

1.7 Delivery aggregators

Uber Eats + PickMe integration built as a **swappable mock** (incoming queue, accept/lifecycle, 86 items, per-location config). Real partner sandboxes pending credentials. 🟡

1.8 CRM / customers

Customer master + categories, per-customer (or category) default discount, **credit customers (A/R)** with limit + balance + AR receipts + ledger, visit history, per-product customer pricing, advance/suspended payments. ✓

1.9 Loyalty

Earn points on settle (configurable rate), redeem as a “Points” tender, balance + transaction ledger, **tiers** (earn multiplier + tier discount), **points expiry**, **loyalty cards** (by-card lookup). Expiry-reminder *delivery* needs a notifications channel (follow-on). ✓

1.10 Promotions

Schedulable (date / day / happy-hour window), order-type scoped, **product-discount**, **bill-value (spend & save)**, auto-applied + snapshot; BOGO/bundle/bank-BIN. ✓

1.11 Reporting

VAT return, sales summary (KPIs / per-day / top items), HQ outlet rollup, **sales register**, **item sales**, **stock balance**, **shift settlement**, **promotion usage**, **food costing** (dish cost vs sell price + GP%), **bin card** (per-product movement ledger), **budget vs sales** (per-outlet monthly). ✓ *(a unified inventory_movements table to simplify the bin card is the only follow-on)*

1.12 Shifts & cash control

Open/close cashier shifts, opening float, cash-up **Z-report** + variance, billing gated on an open shift, end-shift reconciliation of open bills, **month-end period lock**. ✓

1.13 People, access & audit

Users + 5 RBAC roles (Owner/Manager/Cashier/Kitchen/Accountant), **granular role permissions** (max-discount %, void/comp), **function-level (per-screen) access**, **manager-PIN override**, append-only **activity/audit log** (who did what, when). ✓

1.14 Authentication

Magic-link (email) **and staff PIN login** (username + PIN, PBKDF2-hashed, logout) for emailless POS staff; rotating JWT + refresh tokens; stale/closed-tenant tokens → 401 (re-auth). ✓

1.15 Platform

Multi-tenant SaaS (control plane + DB-per-tenant + RLS), **tenant auto-provisioning** at signup, configurable document prefixes/branding, multi-outlet (HQ + N locations, branch switching). ✓

1.16 Printing

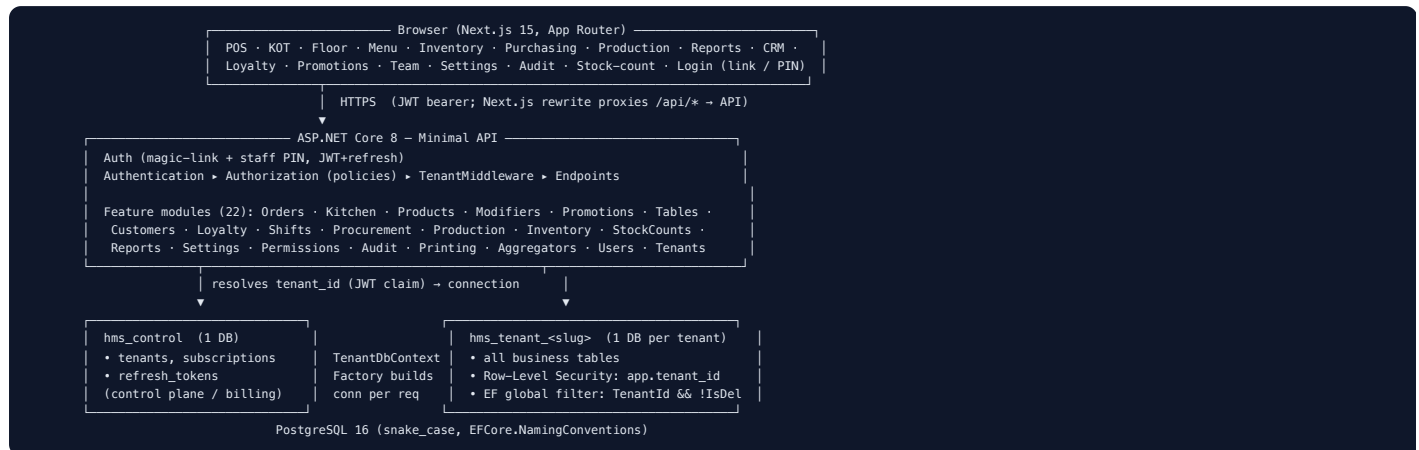
Print Agent design + print-jobs queue for venue receipt/KOT printers. 🟡

Not built (genuine gaps)

- **#67 Gift cards / vouchers** — voucher master, issue/redeem, spend-get-voucher.
- **#73 GL / accounting export** — journal posting by cost-centre, AP/supplier payments, expenses, petty cash (beyond the VAT return).
- **#75 Catering / banquet** — rooms/rates, events, meal types, own-fleet delivery.
- **#74 Handheld waiter app** — a dedicated installable tablet app (a `tab-ordering` web screen exists; native/PWA packaging is pending) and **#57 GRN mobile PWA**.

-  **Serial / warranty tracking** — intentionally out of scope (not F&B).

2. System architecture



Key decisions - DB-per-tenant + RLS. Each tenant has its own database (`hms_tenant_<slug>`); a control DB (`hms_control`) holds tenant/billing rows + refresh tokens. `TenantDbContextFactory` reads the `tenant_id` JWT claim, looks the tenant up in control, and builds a per-request connection from a template (`{tenant_db}` placeholder). Defence-in-depth: Postgres **RLS** (`app.tenant_id`) **and** an EF global query filter (`TenantId == current && !IsDeleted`). - **Minimal-API feature modules.** Each domain area is a `Map...Endpoints()` extension; services (`OrderService` , `ShiftService` , `PermissionService` , ...) hold the logic. - **Auth.** Magic-link (email) or staff PIN (username + PBKDF2 PIN, attempt-lockout); both mint an HS256 JWT + a rotating refresh token. Authorization policies: `Owners` , `BackOffice` , `SupplyChain` , `Operations` , `KitchenView` , `PlatformAdmin` . - **Money/tax integrity.** Settlement decrements stock + records payment in one transaction; the compound tax engine + VAT invoice numbering are server-side and configurable. - **Tenant provisioning.** Signup runs the migration set + a baseline seed against a fresh tenant DB.

Tech stack | Layer | Tech | |—| | Web | Next.js 15 (App Router, React, TypeScript, Tailwind — Stitch design system) | | API | ASP.NET Core 8 Minimal API, EF Core 8, Npgsql | | DB | PostgreSQL 16 (snake_case), DB-per-tenant + RLS, raw SQL migrations (0001–0049) | | Auth | JWT (HS256) + rotating refresh, magic-link, staff PIN (PBKDF2) | | Tests | xUnit integration (real Postgres) + Vitest + Playwright E2E + CI |

3. Deployment architecture

Two shapes are documented (full runbook: `docs/deploy.md`).

A) Incubator — bare-metal (LIVE today)

URL: `https://hms.retailit.lk` · shared RIT incubator VM (Ubuntu, alongside Retail Pulse). System Postgres + nginx + systemd + Let's Encrypt (certbot).



- `hms-api` and `hms-web` run as **systemd services** bound to localhost; **nginx** terminates TLS and reverse-proxies by path; **Postgres 16** is local (control DB + one DB per tenant).
- ⚠️ Unrelated to the legacy RIT servers `MF-SW-LP24 / 161.97.172.35` — those are never touched.

B) Portable — Docker Compose (own server / staging)

Self-contained stack in `docker-compose.yml` : **postgres** → **bootstrap** (one-shot: create + migrate `hms_control`) → **api** (ASP.NET Core 8) → **web** (Next.js) → **caddy** (reverse proxy + auto-TLS). Secrets via `./.env` (`infra/gen-secrets.sh`). Brings HMS up on a dedicated box with one command.

C) Local development

`make dev` runs the API (`:5000`) + web (`:3000`) against a local Postgres. `make reset` drops/recreates + applies all migrations + seeds the demo tenant; `make seed-demo` adds demo customers/promotions/PO+GRN/orders/PINs/loyalty.

Secrets posture: JWT signing key + master encryption key come from the environment/secret store (prod fails fast if the secret is the dev default); aggregator merchant credentials are AES-GCM encrypted in the DB (not env).

4. Quality

- **Integration tests** run against a real ephemeral Postgres (control + tenant DBs, real migrations, RLS) through the actual HTTP pipeline — ~198+ green.
 - **Unit** (Vitest) + **E2E** (Playwright) + **CI**.
 - Append-only **audit log** records sensitive actions (settle, void, discount, shift open/close, permission/PIN changes, stock-count post) with the acting user + role.
-

5. Outstanding (not code)

- **Rotate the leaked legacy SA passwords** on the live RIT servers (highest priority, ops action).
- Apply for **Uber Eats + PickMe partner sandboxes** (integration is mock-ready).
- Internal sign-off on repo destination · pilot/reconciliation with a real outlet.

See `docs/backlog.md` for the detailed legacy→new parity register and the remaining items (#67 gift cards, #73 GL export, #75 catering, and the v1 follow-ons).

Backlog — RIT HMS

Prioritised, living backlog. Status as of **2026-06-04**. IDs match the working task tracker. See `sprint-plan.md` for the sprint grouping.

Legend

P0 ship-blocker for live SaaS · P1 high value · P2 nice-to-have ·  parked ·  blocked on RIT  done ·  partial ·  not started

Remaining work — done vs not done (current)

Legacy **feature parity is complete** and the v1 follow-on tail is cleared. What's left is the go-live infra (P0), net-new P2 modules, small follow-on remainders on shipped modules, and scale hardening. Detail for each ID is in the tables below.

Blocked on RIT (can't action in code)

-  **#14 Rotate the leaked SA passwords** on the live servers — **P0, URGENT**.
-  **#9 Aggregator access** —  PickMe real API live + sandbox-validated;  PickMe **LIVE key** (on UAT sign-off) +  **Uber Eats sandbox**.
-  **#13 Pilot** at one outlet + daily reconciliation over two live shifts — P1.
-  **#7 Internal sign-off** on repo destination — P2.

P0 — the only go-live blocker left

-  **Deploy + ops** — hosting, automated prod migration runner, backups, error tracking, health dashboards.

P2 — whole modules NOT built

-  **#67 Gift cards / vouchers** ·  **#73 GL / accounting export (journal/AP/expenses/petty cash)** ·  **#74 Handheld waiter PWA** ·  **#75 Catering / banquet** ·  **#76 POS depth** (covers ; steward/tips, multi-currency tender, tour-agent ) ·  **#53b Product CSV bulk import** (CRUD , importer ) ·  **#57 GRN mobile PWA** (parked) ·  **Manager read-only mobile dashboard**.

Follow-on remainders on shipped modules

-  **#16 Print agent binary** (queue ) + per-agent token + POS “enqueue vs browser-print” toggle.
-  **#66 expiry-reminder delivery** (needs a notifications channel; expiring-soon endpoint )
-  **#65 bank-BIN promos** (needs card-BIN capture at settle).
-  **#72 unified inventory_movements ledger** (bin card works without it).
-  **#77 day-end-across-shifts rollup** (month-end lock )
-  **#71 document state-machine / change log** (per-screen perms )

Scale / hardening (surfaced building v1 — fine for a single-node pilot)

-  **Realtime KDS multi-instance backplane** (SSE is in-memory; add Postgres LISTEN/NOTIFY or Redis for >1 API node).
-  **PickMe poller leader-election** (so a tenant isn't polled by every node against the 30/min cap).
-  **Provisioning background job** (runs inline today) + **prod secret store** for JWT signing / master keys.

Low / backlog (won't block anything)

-  Scale/weighed items (weight-per-unit) ·  Sinhala / secondary-language names + reference codes ·  reorder qty + period (reorder level exists) ·  requisition → PO merge + reorder-level PO suggestions ·  supplier ↔ product link (preferred supplier / agreed cost) ·  receipt-print short name ·  per-product price floor/ceiling ·  GRN in-place correcting edit + back-dated receipt ·  PO/GRN summary-detail + Excel export ·  sales targets ·  kitchen-instruction templates.

Sprint A — procurement & product parity (DONE)

ID	Item	Status
#49	PO + GRN purchasing unit → stock conversion	

ID	Item	Status
#50	PO line/header discounts + freight (landed cost)	✓
#51	GRN free items + supplier invoice #	✓
#52	PO send/cancel/edit + GRN void/reverse	✓
#56	PO approval (maker-checker) + supplier returns/PRN + currency/delivery/terms	✓

Sprint B

— customer-facing depth (DONE)

Product master is now at full legacy-parity-plus. Next up: **Sprint C (live-SaaS P0s)**.

ID	Item	Pri	Notes
#54	Modifiers / add-ons (+ POS wiring)	✓	groups/items + attach + POS picker + line pricing.
#55a	Kitchen / printer routing	✓	Stations CRUD + per-product routing + KOT split per station + printer name (for #16 print agent). Menu dropdown + stations manager.
#55b	Serving-size variants (cup/pot, S/M/L)	✓	One product, several sellable sizes — each its own price + printed label. POS size-picker, Menu sizes editor. (v1: price/name; stock decrements by qty.)
#55c	Per-location pricing / price levels	✓	Named levels (Dine-in/Delivery/...), per-product overrides, auto-select by order type (delivery markup), explicit override. Menu price-by-level matrix + levels manager.
#55d	Per-product tax class	✓	VAT base now honours per-product taxability — standard / zero-rated / exempt drop in/out of the VAT base; mixed baskets taxed correctly. (Org-wide stacked/compound taxes already configurable via the bill-level engine, #25.) Menu tax-treatment dropdown.
#53b	Product CSV bulk import	P2	CRUD + Menu UI  ; bulk import for onboarding still a stub.

Soon (Sprint C

— make it a live SaaS)

ID	Item	Pri	Notes
#17/#20	Tenant auto-provisioning	✓	Signup → ProvisionAsync : CREATE DATABASE → run every tenant migration → seed baseline. Runs inline; a background job is the only scale follow-on.
—	Auth hardening	✓	Magic link returned only in Development; tenant list PlatformAdmin -gated; refresh/logout + rotating refresh tokens; secrets via master key. (Prod: load signing/master keys from a real secret store.)
—	Deploy + ops	P0	THE remaining go-live blocker: hosting, automated migration runner in prod, backups, error tracking, health dashboards.
#26	Outlet switcher + location types	✓	HQ rollout  + Owner/Manager branch switcher in the sidebar (drives POS/floor/reports).  outlet CRUD + types/capabilities (outlet / central_kitchen / warehouse / head_office · can-sell/produce/stock) via Settings → Outlets.

Later (Sprint D–E)

ID	Item	Pri	Notes
#16	Print agent (KOT + bills, thermal)	●	✅ server-side queue built: <code>print_jobs + /api/v1/print</code> (enqueue · poll · ack), contract in <code>docs/print-agent.md</code> . ↳ remaining: the venue agent binary , a per-agent token, and a POS “enqueue instead of browser-print” toggle.
#57	GRN mobile PWA	P	Parked by request. Foundation kept (barcode field + viewport). Resume: <code>/m/grn</code> installable app, scan, receive vs PO + ad-hoc.
—	Manager mobile dashboard (read-only)	P2	
#13	Daily reconciliation report + pilot at one outlet	P1	Aggregator settlement vs sales; two live shifts.

Legacy-parity modules NOT yet built (full system gap analysis, 2026-06-02)

From a code-level inventory of the legacy `legacy/HospitalityManagement` app (controllers + `RIT.HMS.Domain/BLL` entities) diffed against the new system. These are **whole modules / capabilities the legacy has that the rewrite does not yet** — the real remaining parity surface. ✅ full · ● partial · ❌ none.

ID	Module	New status	Pri	Notes / legacy mapping
#65	Promotions engine	● v1+	P1	✅ schedulable (date/day/happy-hour window) + order-type-scoped product-discount , bill-value (spend & save) , buy-X-get-Y (BOGO) and bundle/combo — all auto-applied at the till, netted on the bill + snapshotted (<code>order_promotions</code>); admin UI (per-type rule editors) + POS line. ✅ lowest-price (3-for-2) + customer-segment scope. ↳ follow-on: bank-BIN (needs card-BIN capture at settle). Legacy: <code>InvPromotion*</code> , <code>RstPromotions</code> .
#66	Loyalty program	● v1	P2	✅ v1 (built on CRM #70): org earn rate (points per LKR) + redeem value ; points earned automatically on settle for the attached customer; redeemed at the till as a “Points” tender (bounded by balance); per-customer balance + lifetime + transaction ledger in the CRM drawer; Settings config toggle. ✅ tiers (earn multiplier + tier discount by lifetime points) + points expiry (inactivity window, lazy + sweep endpoint) — tier CRUD + expiry config in Settings. ✅ loyalty cards (card no. + by-card lookup) + expiring-soon endpoint. Legacy: <code>InvLoyaltyTransaction</code> , <code>LoyaltyCustomer</code> . ↳ follow-on: expiry-reminder <i>delivery</i> (needs a notifications channel).
#67	Gift cards / vouchers (Phase 2)	❌	P2	Voucher master + books, purchase/GRN/transfer/cancel, spend-get-voucher promo, POS redemption. Legacy: <code>InvGiftVoucher*</code> , <code>GiftVoucher*Controller</code> .
#68	Table / floor management + reservations	● v1	P1	✅ v1: per-outlet tables (code/seats/area), Floor screen grid with live open/occupied (derived from open orders), tap → open/resume a bill at that table; reservations list + lifecycle (book/seat/cancel/no-show). POS <code>?table=</code> deep-link + <code>orders.table_id</code> . ✅ visual X/Y floor plan — drag-to-arrange Plan view on the Floor screen (<code>pos.x/pos.y</code> , live open/occupied, tap to open/resume). ↳ follow-on: chairs, shapes. Legacy: <code>TableMaster</code> , <code>ChairMaster</code> .
#69	Split / merge bill + table transfer	● v1	P1	✅ v1: merge another open tab into the current bill (source voided), transfer to another table (blocks occupied), POS table picker on “+” (billing selects a live floor table; resume occupied). ✅ item-level split bill : move chosen item quantities (incl. partial) onto a new bill at the same table (<code>/orders/{id}/split</code> + POS Split modal).

ID	Module	New status	Pri	Notes / legacy mapping
#70	CRM / customers	● v1	P1	✓ v1: customer master + categories, per-customer (or per-category) default discount auto-applied when attached at the till, credit customers (AR) — credit limit + running balance, charge-to-account (“Credit”) tender bounded by the limit, AR receipts that pay the balance down, ledger (visit history + receipts). POS: attach/quick-add a customer on the bill; Customers screen (CRUD + categories + ledger drawer). ✓ per-product contract pricing (customer/category price override resolved at the till) + AR statements (opening + running balance + closing for a period) — both in the CRM drawer. ✓ advance/deposit payments (<code>advance_balance</code> + ‘advance’ tender); suspended bills are covered by held/open orders + recall. Legacy: <code>Customer</code> , <code>CustomerCategory</code> , <code>CustomerDiscounts</code> , <code>CustomerPreviousVisit</code> .
#71	Granular permissions + approvals	● v1	P1	✓ v1: configurable per-role POS limits (<code>role_permissions</code>) — max-discount %, can-void, can-comp/no-sale — enforced server-side on the discount & void endpoints by the caller’s role (Owner bypasses; unconfigured roles stay permissive so nothing breaks until tightened). Owner-only Role permissions matrix on the Team screen. ✓ manager-PIN override: an Owner/Manager PIN authorises an over-limit discount or a void at the till (verified server-side, audited as “approved by ...”). ✓ function-level (per-screen) access (<code>role_screen_access</code>) — owners hide whole screens per role; sidebar gates + Team matrix. Legacy: <code>CashierPermission</code> , <code>SysUserGroupPermission</code> . ↳ follow-on: doc state machine (<code>DocStatusChangeLog</code>).
#72	Reports library	● v1	P1	✓ now: VAT return · sales summary (KPIs/per-day/top items) · HQ outlet rollup · sales register (transaction list + tender + totals) · item sales (usage/best-sellers) · stock balance (on-hand + value, as-at) · shift settlement (cash-ups + variance) · promotion usage. All on the Reports screen with date/outlet filters. ✓ food costing (dish cost from production roll-up / recipe estimate vs sell price + GP%) · ✓ bin card (per-product movement ledger from GRN/sales/wastage/adjustment/transfer/production with running balance). ✓ budget vs sales (per-outlet monthly target vs actual). ↳ follow-on: a unified <code>inventory_movements</code> table to simplify the bin card. Legacy: <code>ReportController</code> , <code>BLL_Reports</code> .
#73	GL / accounting export	●	P2	Have: VAT return. Missing: journal posting/export by cost-centre, payment terms, supplier payments / AP, expenses, petty cash, cheque/bank tracking, month-end close. Legacy: <code>ImportJournalDetails</code> , <code>BLL_Journal</code> , <code>PaymentTerm</code> .
#74	Handheld waiter order-taker	✗	P2	Legacy STOS tablet app: waiter takes orders at table → KOT → main POS. Legacy: <code>HMSOrderTaker</code> project, <code>STOS_TabOrder*</code> . (New PWA shell could host this + #57 GRN.)
#75	Catering / banquet	✗	P2	Rooms + room types + rates, events, meal types, own-fleet delivery (persons + vehicles), catering tax mode. Legacy: <code>RoomMaster</code> , <code>RoomType(Rate)</code> , <code>Event</code> , <code>MealType</code> , <code>DeliveryPerson</code> , <code>Vehicle</code> , <code>CateringMood(Tax)</code> .
#76	POS depth: covers, steward, tips, multi-currency	●	P2	Covers (adults/children) per bill, steward/waiter attribution + tips/gratuity, multi-currency tender (currency + FX rate), tour-agent commission. Legacy: <code>TransactionDet.StewardID/NoOfAdults/NoOfChild</code> , <code>PaymentDet.CurrencyCode/CurrencyRate</code> , <code>TourAgentId</code> .
#77	Day-end / month-end + stock count + audit log	● v1	P1	✓ now: audit / activity log (append-only: settle/void/discount/shift open-close/permission changes + stock-count post, with actor+role; filterable screen) · physical stock count (open a sheet → snapshots on-hand per stocked product → enter counts → post writes the variance back to stock + stamps <code>last_counted_at</code>; Stock-count screen). Have: shift Z-report. ✓ month-end period lock (close books through a date; settle + GRN/return/production reversals into a closed period are blocked; close=BackOffice / reopen=Owner, audited). ↳ follow-on: formal day-end across shifts rollup. Legacy <code>TransactionLog</code> / <code>DocStatusChangeLog</code> / <code>L0G*</code> .
#78	Recall settled bill + reprint receipt	●	P1	✓ now: Print on the POS prints the live bill/tax-invoice (80mm thermal receipt via <code>/orders/{id}/invoice</code>), gated on items (no more blank-bill print). ✓ recall a settled bill (<code>/orders/settled</code> search) + reprint a duplicate (<code>reprint_count</code> , marked “REPRINT · COPY N”, audited) via a POS Recall modal. Legacy: duplicate-bill print + bill reprint permission.

Already at full legacy parity (✓): POS billing + multi-tender settlement, KOT + per-station/printer routing, shifts + Z-report, order resume/suspend-recall, products/categories/units + UoM conversion, modifiers/sizes/price-levels/tax-class, suppliers/PO/GRN/PRN, transfers/wastage/adjustments, production (BOM + parity), compound tax/VAT, document numbering/prefixes. **New does more than legacy:**

multi-tenant SaaS + RLS + auto-provisioning, JWT auth + refresh, Uber/PickMe aggregator integration, weighted-avg cost roll-up into finished goods.

Blocked on RIT

ID	Item	Pri	Notes
#14	Rotate leaked SA passwords on live servers	P0	URGENT — compromised regardless of this rewrite.
#9	Apply for Uber Eats + PickMe sandboxes	P1	 PickMe: real API live (POS API v1.4.7) — per-outlet X-API-KEY (encrypted), <code>/joblist</code> poller (pull/mirror model, gated <code>Aggregators:PickMe:PollingEnabled</code>), order ingestion (items by <code>ref_id=SKU</code> , <code>options/sp_ins</code> →notes, prepaid settle, status mirror, cancel→void), menu/price/availability push (<code>update/item/ref</code>), live sandbox validated end-to-end. ↳ Uber Eats still mock (needs its sandbox). ↳ PickMe LIVE key on UAT sign-off.
#7	Internal sign-off on repo destination	P2	

Legacy → New parity register (detail: product / PO / GRN / production)

Complete record from the legacy-vs-new gap analyses (product master, PO, GRN, production). High-severity items are already top-line above; this captures the **remaining** medium/low items so none are lost. **Legend:**  = done · ↳ Sprint A/B = **delivered** (those sprints are complete) · ↳ Sprint C / reporting / backlog = still planned.

Product master

Gap	Sev	Status
Update / delete + create UI	High	 done (#53)
Modifiers / add-ons  #54) · kitchen routing  #55a) · serving sizes  #55b) · per-location pricing  #55c) · per-product tax class  #55d)	High	 Sprint B complete (#54 / #55a–d)
Supplier ↔ product link (preferred supplier, last/agreed cost)	Med	↳ Sprint A
Receipt-print name (short bill name vs long menu name)	Med	↳ Sprint B
Per-product discount rules + price floor/ceiling (max/fixed %, min/max)	Med	↳ Sprint B
Reorder quantity + period (only reorder <i>level</i> today)	Med	↳ Sprint C
Pack / bundle / combo items	Med	↳ Sprint B
Scale item / weight-per-unit (weighed goods)	Low	backlog
Sinhala / secondary-language name · reference codes (RefCode01/02)	Low	backlog
Sales targets (qty/period/type) · kitchen instruction templates	Low	backlog
Open item · batch/expiry	—	 (POS open item; GRN batch/expiry)

Purchase Orders

Gap	Sev	Status
Unit conversion · discounts/charges · approval · cancel/edit · currency · terms	High/Med	 done (Sprint A — #49/#50/#52/#56)
Per-line / multi-tax + tax-on-tax	Med	 product-side tax class  (#55d); per-PO-line multi-tax not yet

Gap	Sev	Status
Separate delivery vs PO location (+ delivery address)	Med	✅ done (PO delivery_location_id + delivery_address)
Reorder-level PO suggestions / requisition → PO merge	Med	↳ Sprint C
Month-end period lock (block posting into closed periods)	Med	✅ done (#77 — 0041_period_lock ; settle/GRN/return/production into a closed period are blocked)
Free qty on PO line · validity/expiry dates · event tag · requested-by · reference no	Low	backlog (free qty exists on GRN)
PO summary/detail reports + Excel export	Low	↳ reporting

GRN

Gap	Sev	Status
Unit conversion · free items · line discount · supplier invoice # · void/reverse	High	✅ done
Supplier returns / debit note (PRN) + input-VAT reversal · approval	High/Med	↳ Sprint A (#56)
Recipe/menu cost roll-up on receive (recompute dish cost from new avg)	Med	↳ Sprint A/B
GRN in-place edit (void exists; correcting edit doesn't)	Med	↳ Sprint A
inventory_movements ledger row on GRN (transfers/wastage log; GRN mutates stock directly)	Med	↳ Sprint A (audit consistency)
Month-end lock · back-dated receipt · GRN summary/detail reports · price-level on receive	Low	backlog / reporting
Serial / warranty tracking	—	Won't do (not F&B)

Production

All legacy capabilities reached: BOM + **UoM conversion**, custom/ad-hoc, draft→post→**void**, multi-product document, multiple recipes per output unit, inter-location. Legacy in-place “edit production” ≈ our **void + re-produce**. New does better: weighted-avg cost roll-up into the finished good (legacy didn't).

Done (highlights)

Foundation (cloud rewrite, multi-tenant + RLS) · **JWT auth enforced + RBAC** · POS (keypad/discount/multi-tender/open items) · KOT · **compound tax/VAT engine** · **shifts + cash-up Z-report** · suppliers/PO/GRN (receive integrity) · transfers/wastage/adjustments · **production (BOM + UoM conversion + full legacy parity)** · delivery aggregators (mock) · **product CRUD + Menu screen** · reports (VAT return, sales, **HQ outlet rollup**) · user/team management · configurable prefixes/branding/tax-certs · **menu depth: modifiers, kitchen routing, serving sizes, price levels, tax class** · **auth hardening (refresh/logout, prod-secret guard, admin-gated tenants)** · **tenant auto-provisioning** (signup → DB) · **real PickMe POS API** (poll + menu push) · **loyalty tiers + expiry + cards** · **month-end period lock** · **visual floor plan** · **per-product contract pricing + AR statements + advances** · **function-level perms** · **food costing / bin card / budget-vs-sales** · **recall + reprint** · **print-job queue**. Tested: 233 integration · 36 unit · ~24 E2E · CI.